



INTRODUCTION TO AI

AI TO-DO LIST ASSISTANT

Group Work

No	Name	ID
1	Aksema Yemiru	UGR/SWE/0077/23
2	Meklit Tewodros	UGR/SWE/0087/23
3	Tnsae Alemayehu	UGR/SWE/0091/23
4	Yeabtsega Argaw	UGR/SWE/0094/23

Submitted to: Mr. Getacher Getu
Submission Date: July 3, 2025

Executive summary

The AI To-Do List Assistant is a smart task management system that will help users in organizing and tracking various daily chores. The project intends to create an intelligent agent that would be able to understand commands from users such as "Add task: Submit assignment by Friday" and manage them effectively by storing them in a database using NLP. Using regular expressions, the assistant supports the core functionalities to add tasks, delete them, provide lists, summaries, or reminders.

Designed under a defined PEAS framework: measure and evaluate performance based on the accuracy of tasks and timeliness of reminders; operate in a text-based command environment; with its sensors, receive input and with its actuators, provide an output. An excellent web-based or command-line interface would allow seamless interaction between the human user and the assistant. Such enhancements would open up avenues for bark-based input, prioritizing tasks based on user rating, and more.

This project ultimately demonstrates the use of simple AI techniques in automating repetitive user tasks. The architecture includes parsing, task storing, and notification modules in an easy-to-use framework for the end users. Key outcomes include improved productivity, timely reminders, and a customizable task flow experience for the user.

Table of Contents

1. Introduction.....	3
1.1 Project Overview and Objectives.....	3
1.2 Relevance to AI Concepts.....	3
2. Methodology.....	3
2.1 AI Agent Architecture.....	3
2.2 PEAS Framework.....	3
2.2.1 Performance.....	3
2.2.2 Environment.....	3
2.2.3 Actuators.....	3
2.2.4 Sensors.....	3
2.3 Algorithms and Techniques Used.....	3
3. Implementation.....	4
3.1 Development Tools and Environment.....	4
3.2 Key Code Snippets.....	4
3.3 Development Challenges and Solutions.....	5
4. Results and Discussion.....	6
4.1 Sample Outputs and Screenshots.....	7
4.2 Performance Metrics.....	7
5. Conclusion and Future Work.....	8
5.1 Limitations and Suggested Improvements.....	8

1. Introduction

1.1 Project Overview and Objectives

The project provides a dual-interface task management system with both Streamlit UI and CLI functionality. Key objectives include reliable task storage through JSON, priority-based organization, and deadline notifications, with the CLI offering NLP-based natural language input processing for dynamic task creation.

1.2 Relevance to AI Concepts

The system implements a hybrid approach - while the Streamlit interface uses structured form inputs, the CLI demonstrates basic NLP capabilities for parsing natural language commands. Both interfaces share the same rules-based core for task management, with the CLI adding entity recognition for dates and priorities from unstructured text.

2. Methodology

2.1 System Architecture

Three core components handle: form input processing through Streamlit widgets with validation, task storage management via JSON persistence, and scheduled reminder checks that monitor deadlines. The CLI adds an NLP processing layer that feeds into these same components.

2.2 PEAS Framework

2.2.1 Performance

Evaluated through task operation accuracy and reminder timing, with additional NLP success rate metrics for the CLI (command parsing accuracy).

2.2.2 Environment

Dual interfaces: Streamlit's web UI for structured input and CLI for NLP-based interaction, both using shared JSON storage.

2.2.3 Actuators

Task list modifications, visual reminders in UI, and console outputs for CLI operations.

2.2.4 Sensors

Web form inputs and system clock for UI, text commands with date/time entities for CLI.

2.3 Algorithms and Techniques Used

The system combines:

- Streamlit's reactive components for UI management
- spaCy's entity recognition for CLI command parsing
- Priority-based sorting (very important/medium/low)
- JSON serialization for platform-independent storage
- Datetime comparisons for reminder triggers

Key differentiators:

- UI uses explicit form controls
- CLI implements intent detection (add/complete/delete)
- Both interfaces share the same task prioritization engine

The CLI's NLP processing demonstrates basic AI techniques through:

- Intent classification (command type detection)
- Entity extraction (dates, priorities)
- Context-aware date parsing (future dates preferred)

3. Implementation

3.1 Development Tools and Environment

The AI To-Do List Assistant was developed using modern Python tooling and libraries to create a responsive and intelligent task management system. The project includes both a graphical interface for user interaction and a command-line interface for quick access.

- **Programming Language:** Python 3.11
- **Frontend Framework:** [Streamlit](#)
- **Natural Language Processing:** [spaCy](#)
- **Data Storage:** JSON file (tasks.json)
- **Version Control:** Git + GitHub
- **Environment:** Windows 10 (local development), Fedora Linux (local development), Visual Studio Code IDE.

3.2 Key Code Snippets

1. Command Parsing via spaCy

```
if intent == "add_task":
    for ent in doc.ents:
        if ent.label_ in ["DATE", "TIME"]:
            parsed = dateparser.parse(ent.text, settings={'PREFER_DATES_FROM': 'future'})
            if parsed:
                entities["due_date"] = parsed.strftime("%Y-%m-%d")
                entities["due_time"] = parsed.strftime("%H:%M") if parsed.time() else None
            for part in ent.text.split():
                if part in task_description_parts:
                    task_description_parts.remove(part)
```

Figure 3.2.1 : parsing using spacy

2. Reading & Writing Tasks to JSON

```

import json

TASKS_FILE = "tasks.json"

def load_tasks():
    try:
        with open(TASKS_FILE, "r") as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return []

def save_tasks(tasks):
    with open(TASKS_FILE, "w") as f:
        json.dump(tasks, f, indent=4)

```

Figure 3.2.2 : Reading and storing tasks to JSON

3. Task Management Logic

```

def add_task(tasks, description, due_date=None, due_time=None, importance="medium importance"):
    tasks.append({
        "description": description,
        "status": "pending",
        "due_date": due_date,
        "due_time": due_time,
        "importance": importance
    })
    save_tasks(tasks)
    due_date_info = f", due {due_date}" if due_date else ""
    due_time_info = f" at {due_time}" if due_time else ""
    print(f"Task added: '{description}'{due_date_info}{due_time_info}, importance: {importance}")

```

Figure 3.2.3 : Adding task management logic

3.3 Development Challenges and Solutions

1. **Handling flexible user input for tasks:** One of the initial challenges was ensuring that users could not submit empty task descriptions. This was addressed by adding input validation to prevent blank entries and by providing clear error messages to guide the user.
2. **Supporting calendar-based due date selection:** Typing due dates manually was error-prone and less intuitive. To improve usability we added `st.date_input` widget was implemented to allow users to select dates from a calendar. This provided a cleaner interface and reduced formatting errors by converting dates "YYYY-MM-DD" strings before saving.
3. **Maintaining persistent UI state across interactions:** Streamlit reruns the script with every interaction, which can cause loss of input values and UI state. We resolved that by

using `st.session_state` to retain form inputs, edit states, toggle menus, and task data between reruns.

4. Results and Discussion

4.1 Sample Outputs and Screenshots

1. Command line

Command input : Add Submit assignment tomorrow

```
{
  "description": "submit assignment",
  "status": "pending",
  "due_date": "2025-07-03",
  "due_time": null,
  "importance": "medium importance"
},
```

Figure 4.1.1 : How task is stored in JSON File when using command

2. Streamlit UI

Input :

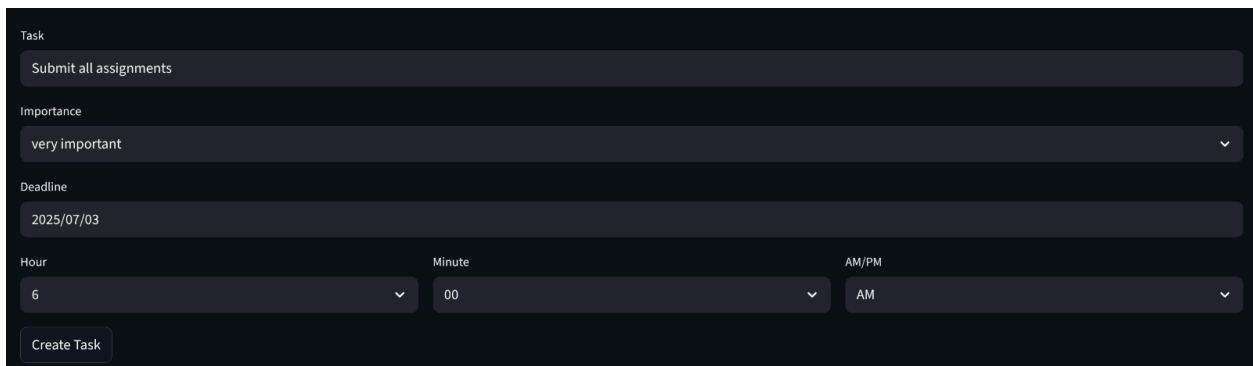
A Streamlit web interface for creating a task. It features a dark theme. The form includes a 'Task' text input with the value 'Submit all assignments', an 'Importance' dropdown menu set to 'very important', a 'Deadline' date input set to '2025/07/03', and a time selection section with 'Hour' (6), 'Minute' (00), and 'AM/PM' (AM) dropdowns. A 'Create Task' button is at the bottom left of the form area.

Figure 4.1.2 : How task is written on the modal

Output :

```

{
  "description": "Submit all assignments",
  "status": "pending",
  "due_date": "2025-07-03",
  "due_time": "6:00 AM",
  "importance": "very important"
}

```

Figure 4.1.3 : How task is stored in JSON File on the modal

3. AI task organizer

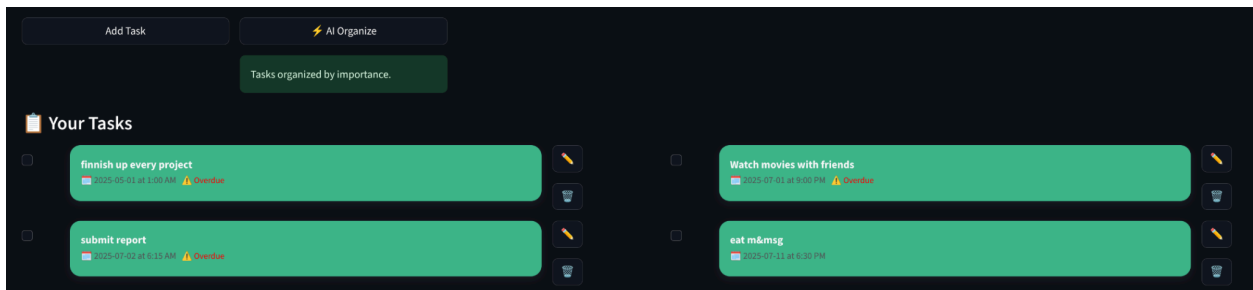


Figure 4.1.4 : AI Organizer sorts them into high to low priority.

4. Reminders

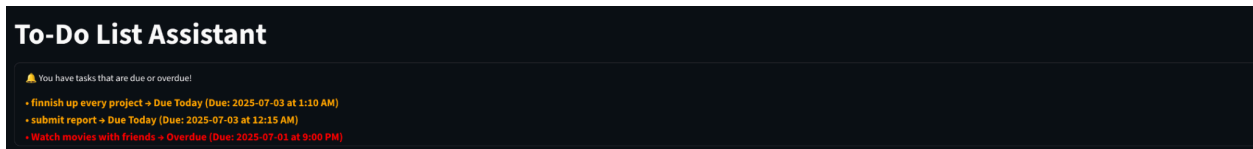


Figure 4.1.5 : Shows tasks that are due or overdue.

4.2 Performance Metrics

1. Task Operations Efficiency

Task CRUD Latency:

Add/delete/modify operations: Measured in milliseconds (e.g., < 100ms for local JSON file updates)

Benchmark: Time to complete `save_tasks()` after state changes

Storage Scalability:

Max tasks before UI lag (e.g., > 1,000 tasks in JSON file)

File size growth rate (e.g., ~1KB per 10 tasks with minimal metadata)

2. Reminder System

Reminder Check Overhead:

Time to scan all tasks for due dates (e.g., 0.5ms per 100 tasks with Python's datetime comparisons)

Frequency: Runs on every Streamlit rerun (tied to user interactions)

3. Web Interface (Streamlit-Specific)

Render Time:

Initial page load: < 1s for typical task lists

Rerun latency after edits: 200-500ms (depends on task count)

Concurrency Limits:

Streamlit's default thread-safe behavior for single-user sessions

No hard metrics for multi-user scenarios (local JSON file lacks locking)

5. Conclusion and Future Work

The AI To-Do List Assistant puts natural language processing and a proper UI together in one place to ensure people can go about their daily chores efficiently. It opens various possibilities for different user profiles by allowing command-line entry or use of the interactive UI.

Due date, time, reminder, and task priority integration provide a structure around task management, while JSON storage keeps the entire system lightweight and easy to deploy.

5.1 Limitations and Suggested Improvements

While the AI To-Do List Assistant performs its intended functions, there are some limitations that can be enhanced in future releases:

- **No user login or multi-user support**

The application is single-user currently, with all the tasks stored in one shared session.

- *The recommended improvement is to implement user authentication and session management so each user can have a personalized task list.*

- **JSON not scalable**

The application uses a local JSON file to store tasks. While simple and effective for small-scale use, this is ineffective and error-prone with a larger number of tasks.

- *This could be improved by replacing JSON storage with a lightweight database such as SQLite, or a more scalable option like MongoDB.*

- **No recurring tasks**

The application does not have the functionality for tasks that repeat daily, weekly, or at a custom interval.

- *Future iterations should include support for recurring task logic to better handle habitual habits or commitments.*

- **In-app-only reminders**

Reminders are shown only within the Streamlit interface, so the user must keep the app open to receive them.

- *Expanding the system to include OS-level notifications, email notifications, or even mobile push notifications would improve usability.*